

Assignment 2 — Interpolation-Based Super-Resolution

Digital Image Processing

Om Kumar Gupta
Roll No. 2023362
IIIT Delhi

September 19, 2026

Contents

1	Objective and scope	2
2	Dataset	2
3	Method: formulation and implementation	3
3.1	Grayscale conversion	3
3.2	Downsampling by exact block averaging	3
3.3	Coordinate mapping	3
3.4	Separable interpolation	3
3.5	Boundary handling and weight normalisation	4
3.6	Interpolation kernels	4
3.7	PSNR	5
3.8	SSIM	5
3.9	Measurement protocol	5
4	Implementation validation	6
5	Results	7
5.1	Downsampled inputs	7
5.2	Super-resolved outputs	8
5.3	Magnified crops	11
5.4	PSNR	13
5.5	SSIM	14
5.6	Runtime	15
6	Discussion	15
6.1	Quality across enlargement factors	15
6.2	Visual differences among the methods	15
6.3	Artefacts	16
6.4	Measured overshoot	16
6.5	Quality versus runtime	17
6.6	Dependence on image content	17
6.7	A case where nearest neighbour wins	18
7	Additional investigation: region-wise error decomposition	18
7.1	Method	18
7.2	Results	19
7.3	Findings	20
8	Additional investigation: cost scaling of the separable engine	21
9	Conclusion	22

1 Objective and scope

Four 512×512 grayscale images are downsampled to 64×64 , 128×128 and 256×256 , then super-resolved back to 512×512 using interpolation kernels of increasing order. Reconstruction quality is measured against the original using PSNR and SSIM, and the computational cost of each method is measured.

Every algorithm in this study is implemented from first principles using NumPy array arithmetic only. No library resize, no library metric and no library convolution routine is used anywhere in the pipeline. Grayscale conversion, block-average downsampling, the separable interpolation engine, all six interpolation kernels, PSNR, SSIM (including its Gaussian windowing) and the Sobel operator used in Section 7 are all written out below and implemented directly. External libraries are used only to read the TIFF files from disk and to draw the figures.

Table 1: Super-resolution settings.

Input size	Output size	Enlargement factor
64×64	512×512	$8\times$
128×128	512×512	$4\times$
256×256	512×512	$2\times$

2 Dataset

Four images were taken from the Miscellaneous volume of the USC-SIPI image database (<https://sipi.usc.edu/database/database.php?volume=misc>), selected to span distinct content types so that the dependence of the results on image content could be examined.

Table 2: Selected images. All converted to 8-bit grayscale at 512×512 before processing.

SIPI id	Name	Native format	Content character
4.2.03	Mandrill / Baboon	24 bpp colour	Dense high-frequency fur texture
4.2.05	Airplane (F-16)	24 bpp colour	Strong clean edges, flat sky
4.2.07	Peppers	24 bpp colour	Large smooth shaded regions
5.2.10	Stream and bridge	8 bpp gray	Mixed texture and structure

Figure 1 — Original grayscale reference images

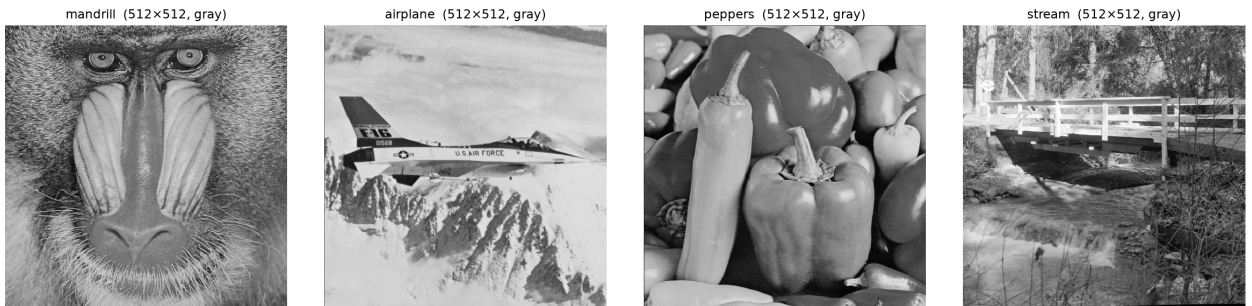


Figure 1: The four original grayscale reference images (512×512 , 8 bpp). These are the ground truth for all PSNR and SSIM computations.

3 Method: formulation and implementation

3.1 Grayscale conversion

Colour images are converted using the ITU-R BT.601 luma weights,

$$Y(m, n) = 0.299 R(m, n) + 0.587 G(m, n) + 0.114 B(m, n), \quad (1)$$

evaluated in `float64` and then rounded and clipped to the 8-bit range,

$$I(m, n) = \min(255, \max(0, \text{round}(Y(m, n))))). \quad (2)$$

Images stored natively as 8 bpp monochrome bypass (1). Conversion is performed before downsampling so that all four images follow an identical path.

3.2 Downsampling by exact block averaging

For a decimation factor $f = 512/N$ with $N \in \{64, 128, 256\}$, each output sample is the mean of a disjoint $f \times f$ block of the reference:

$$I_{\text{LR}}(m, n) = \frac{1}{f^2} \sum_{p=0}^{f-1} \sum_{q=0}^{f-1} I_{\text{ref}}(mf + p, nf + q), \quad 0 \leq m, n < N. \quad (3)$$

This is a box low-pass filter followed by decimation. It was chosen over plain subsampling because plain subsampling would alias high-frequency content — most severely on the mandrill — before any interpolation took place, corrupting every subsequent metric and making the comparison between methods meaningless. Because $512/64$, $512/128$ and $512/256$ are exact integers, every block is fully covered and no partial-coverage weighting is needed. The result is rounded and clipped by (2). The same procedure is used for every image and every size throughout the study.

3.3 Coordinate mapping

Super-resolution maps an N_{in} -sample axis onto an N_{out} -sample axis. Using the half-pixel centre convention, output index i corresponds to the continuous source coordinate

$$x(i) = \left(i + \frac{1}{2}\right) \frac{N_{\text{in}}}{N_{\text{out}}} - \frac{1}{2}. \quad (4)$$

The $\pm \frac{1}{2}$ terms place sample centres rather than sample edges in correspondence, which keeps the reconstruction geometrically centred. Omitting them produces a half-pixel shift that biases every metric downward; verifying (4) was the single most important correctness check in the implementation (Section 4).

3.4 Separable interpolation

For a symmetric kernel $k(\cdot)$ of radius r , the reconstructed intensity at a continuous location (x, y) is

$$\hat{I}(x, y) = \sum_j \sum_i k(x - i) k(y - j) I_{\text{LR}}(j, i). \quad (5)$$

Because the 2-D kernel in (5) factorises as a product of two 1-D kernels, the sum separates into two successive 1-D passes — first along rows, then along columns:

$$\hat{I}(x, y) = \sum_j k(y - j) \left[\sum_i k(x - i) I_{\text{LR}}(j, i) \right]. \quad (6)$$

With $T = \lceil 2r \rceil + 1$ taps per axis, the direct form (5) costs $\mathcal{O}(T^2)$ multiply-accumulates per output pixel while the separable form (6) costs $\mathcal{O}(2T)$. For Lanczos-4, $T = 9$, so separability reduces the

work per pixel from 81 to 18 operations — a factor of 4.5. This prediction is tested against measured runtime in Section 8.

The implementation precomputes, for each axis, the tap index matrix and the weight matrix

$$\Lambda_{i,t} = \lceil x(i) - r \rceil + t, \quad W_{i,t} = k(x(i) - \Lambda_{i,t}), \quad t = 0, \dots, T-1, \quad (7)$$

and applies them to the image as two tensor contractions.

3.5 Boundary handling and weight normalisation

Tap indices falling outside the valid range are clamped to the edge,

$$\Lambda_{i,t} \leftarrow \min(N_{\text{in}} - 1, \max(0, \Lambda_{i,t})), \quad (8)$$

which replicates the border sample rather than introducing a synthetic dark margin. Clamping causes several taps to reference the same sample near a border, so the weights are renormalised to preserve the partition-of-unity property,

$$\widetilde{W}_{i,t} = \frac{W_{i,t}}{\sum_{t'} W_{i,t'}}, \quad \text{so that} \quad \sum_t \widetilde{W}_{i,t} = 1 \quad \forall i. \quad (9)$$

Equation (9) guarantees that a constant image is reproduced exactly by every kernel, which is verified in Section 4. The final output is rounded and clipped by (2).

3.6 Interpolation kernels

All six kernels are interpolating — $k(0) = 1$ and $k(n) = 0$ for every nonzero integer n — so retained samples are reproduced exactly.

Zeroth order (nearest neighbour).

$$k_0(x) = \begin{cases} 1, & -\frac{1}{2} < x \leq \frac{1}{2} \\ 0, & \text{otherwise} \end{cases} \quad r = 0.5, \quad T = 2. \quad (10)$$

The half-open interval avoids an ambiguous tie at exactly $\pm\frac{1}{2}$.

First order (linear).

$$k_1(x) = \begin{cases} 1 - |x|, & |x| < 1 \\ 0, & \text{otherwise} \end{cases} \quad r = 1, \quad T = 3. \quad (11)$$

Second order (Dodgson quadratic, $r = 1$).

$$k_2(x) = \begin{cases} -2|x|^2 + 1, & |x| \leq \frac{1}{2} \\ |x|^2 - \frac{5}{2}|x| + \frac{3}{2}, & \frac{1}{2} < |x| \leq \frac{3}{2} \\ 0, & \text{otherwise} \end{cases} \quad r = 1.5, \quad T = 4. \quad (12)$$

Third order (Keys cubic convolution, $a = -\frac{1}{2}$).

$$k_3(x) = \begin{cases} (a+2)|x|^3 - (a+3)|x|^2 + 1, & |x| \leq 1 \\ a|x|^3 - 5a|x|^2 + 8a|x| - 4a, & 1 < |x| < 2 \\ 0, & \text{otherwise} \end{cases} \quad r = 2, \quad T = 5. \quad (13)$$

The choice $a = -\frac{1}{2}$ makes the reconstruction agree with the Taylor expansion of the underlying signal to third order, which is the standard Keys parameter.

Windowed sinc (Lanczos- A , $A \in \{3, 4\}$).

$$k_A(x) = \begin{cases} \text{sinc}(x) \text{sinc}\left(\frac{x}{A}\right), & |x| < A \\ 0, & \text{otherwise} \end{cases} \quad \text{sinc}(x) = \begin{cases} \frac{\sin \pi x}{\pi x}, & x \neq 0 \\ 1, & x = 0 \end{cases} \quad (14)$$

giving $r = A$ and $T = 2A + 1$, i.e. $T = 7$ for Lanczos-3 and $T = 9$ for Lanczos-4. The kernel is the ideal sinc reconstruction filter truncated by a wider sinc lobe acting as a window.

Kernels (12), (13) and (14) take negative values away from the origin. These negative lobes sharpen edges but allow the reconstruction to exceed the range of the surrounding samples, which is the mechanism behind the ringing and overshoot discussed in Section 6.3. Kernels (10) and (11) are non-negative and, by (9), produce outputs bounded by the convex hull of their input taps — they cannot overshoot.

3.7 PSNR

For an $M \times N$ reference I and reconstruction $\hat{I}$,

$$\text{MSE} = \frac{1}{MN} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} (I(m, n) - \hat{I}(m, n))^2, \quad \text{PSNR} = 10 \log_{10} \left(\frac{L^2}{\text{MSE}} \right), \quad (15)$$

with $L = 255$. Both images are cast to `float64` before the difference is taken, so the subtraction does not wrap around in unsigned arithmetic.

3.8 SSIM

SSIM is computed in the form of Wang et al. (2004), with Gaussian-weighted local statistics over an 11×11 window ($\sigma = 1.5$). The 2-D Gaussian window is separable and is applied as two 1-D passes, exactly as in (6):

$$g(t) = \frac{\exp(-t^2/2\sigma^2)}{\sum_{t'} \exp(-t'^2/2\sigma^2)}, \quad t = -5, \dots, 5. \quad (16)$$

Writing $\mu_x = g * x$ for the windowed mean, the local second moments follow from

$$\sigma_x^2 = g * x^2 - \mu_x^2, \quad \sigma_y^2 = g * y^2 - \mu_y^2, \quad \sigma_{xy} = g * (xy) - \mu_x \mu_y, \quad (17)$$

which avoids a second explicit pass over the window. The local SSIM map is

$$\text{SSIM}(m, n) = \frac{(2\mu_x \mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \quad \begin{aligned} C_1 &= (K_1 L)^2, \quad K_1 = 0.01, \\ C_2 &= (K_2 L)^2, \quad K_2 = 0.03, \end{aligned} \quad (18)$$

and the reported figure is the mean of this map. The first factor of the numerator is the luminance term; the second combines contrast and structure. The contrast term responds directly to local variance, which is the mechanism behind the result in Section 6.7, where a method that preserves local contrast outscores a smoother one on SSIM by a much larger margin than on PSNR.

3.9 Measurement protocol

Runtime covers the interpolation call only — no file I/O, no metric computation, no plotting. Each configuration is executed three times as warm-up and then timed over 15 further runs with `time.perf_counter()`; the reported statistic is the median, which is robust to the occasional scheduling outlier. The full experiment is 4 images $\times$ 3 factors $\times$ 6 methods = 72 reconstructions.

4 Implementation validation

Because every algorithm is written from scratch, three independent checks were run before any result was recorded.

1. The worked example from the assignment prelude. Applying the nearest-neighbour kernel (10) to the 2×2 array given in the handout reproduces the 4×4 result exactly:

$$\begin{bmatrix} 40 & 120 \\ 180 & 220 \end{bmatrix} \longrightarrow \begin{bmatrix} 40 & 40 & 120 & 120 \\ 40 & 40 & 120 & 120 \\ 180 & 180 & 220 & 220 \\ 180 & 180 & 220 & 220 \end{bmatrix}$$

2. Constant reproduction. A uniform image of value 137 upsampled $8\times$ returns exactly 137 at every pixel for all six kernels, confirming the partition-of-unity property (9) including at the borders.

3. Agreement with reference implementations. As an external check only — no library routine is used in the pipeline — the from-scratch results were compared against OpenCV for the three kernels the two implementations have in common. Nearest neighbour, linear and Lanczos-4 agree with OpenCV to the full two decimal places reported, in all twelve image-factor cells. For example, nearest neighbour on mandrill gives 23.35/20.85/19.70 dB and Lanczos-4 on airplane gives 31.90/26.48/22.75 dB under both implementations. Exact agreement across twelve independent cells confirms that the coordinate mapping (4), the boundary handling and the kernel evaluations are all correct; a half-pixel error in (4) alone would have shifted every value by several tenths of a dB.

The quadratic and cubic results differ from the corresponding library routines, as expected: `scikit-image` implements *interpolating B-splines*, which require a recursive prefilter, whereas (12) and (13) are direct convolution kernels. These are different algorithms and are not expected to agree.

5 Results

5.1 Downsampled inputs

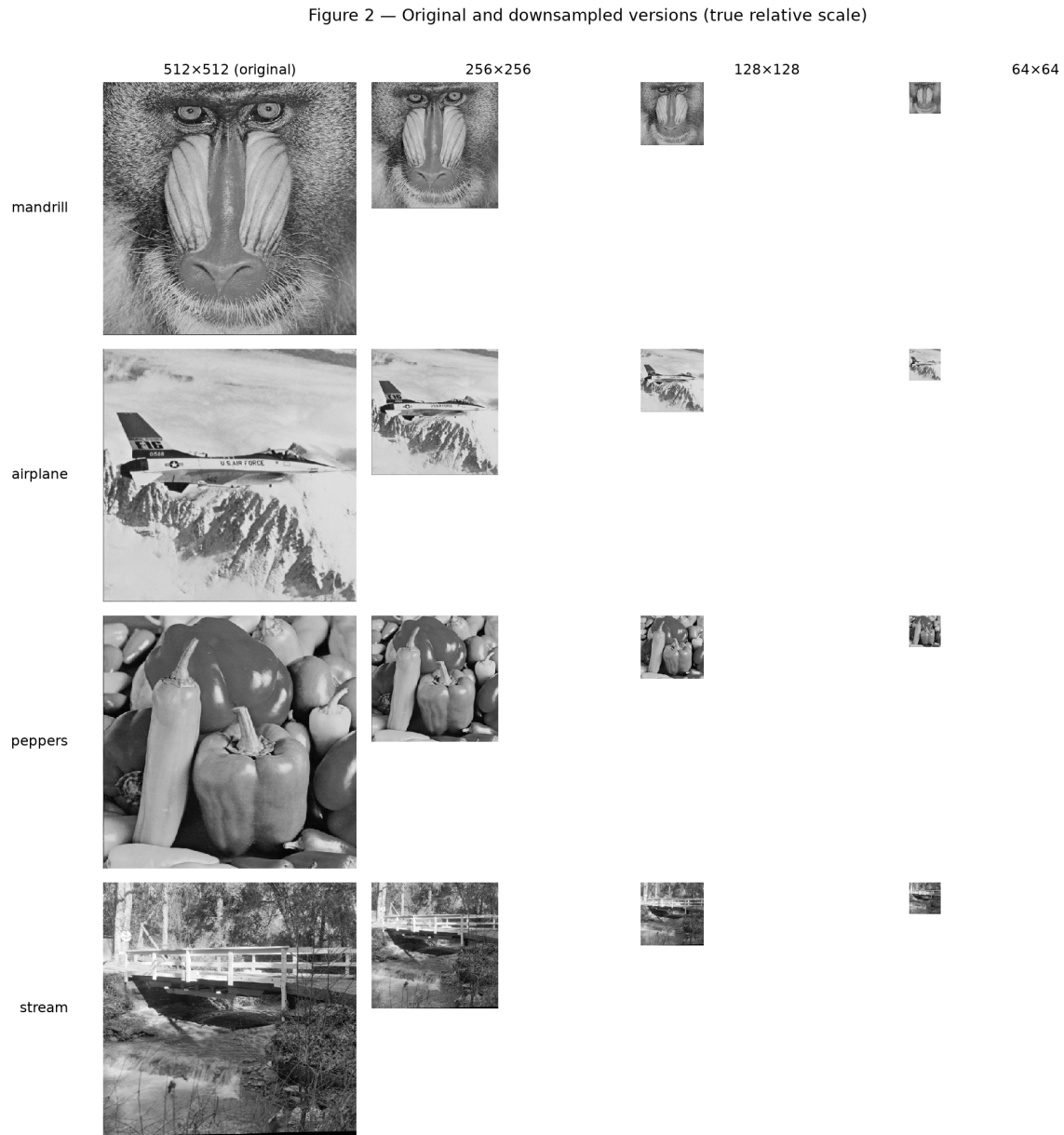


Figure 2: Original and downsampled versions of each image, displayed at true relative scale. The loss of fur texture in the 64×64 mandrill is total.

5.2 Super-resolved outputs

Figure 3 — Super-resolved results: airplane



Figure 3: Super-resolved results for *airplane*: rows are interpolation methods, columns are enlargement factors.

Figure 3 — Super-resolved results: mandrill

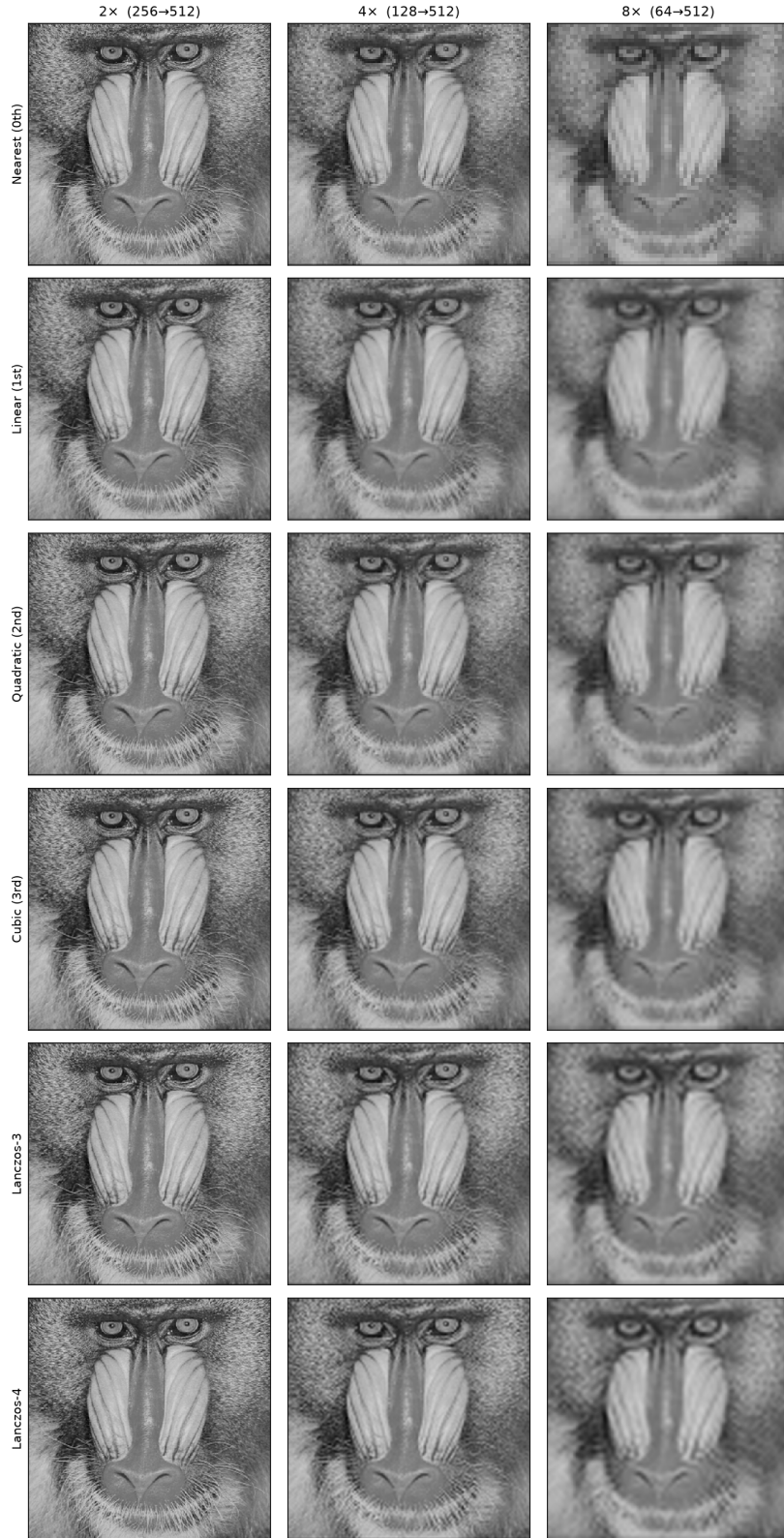


Figure 4: Super-resolved results for *mandrill*.

Figure 3 — Super-resolved results: peppers



Figure 5: Super-resolved results for *peppers*.

Figure 3 — Super-resolved results: stream

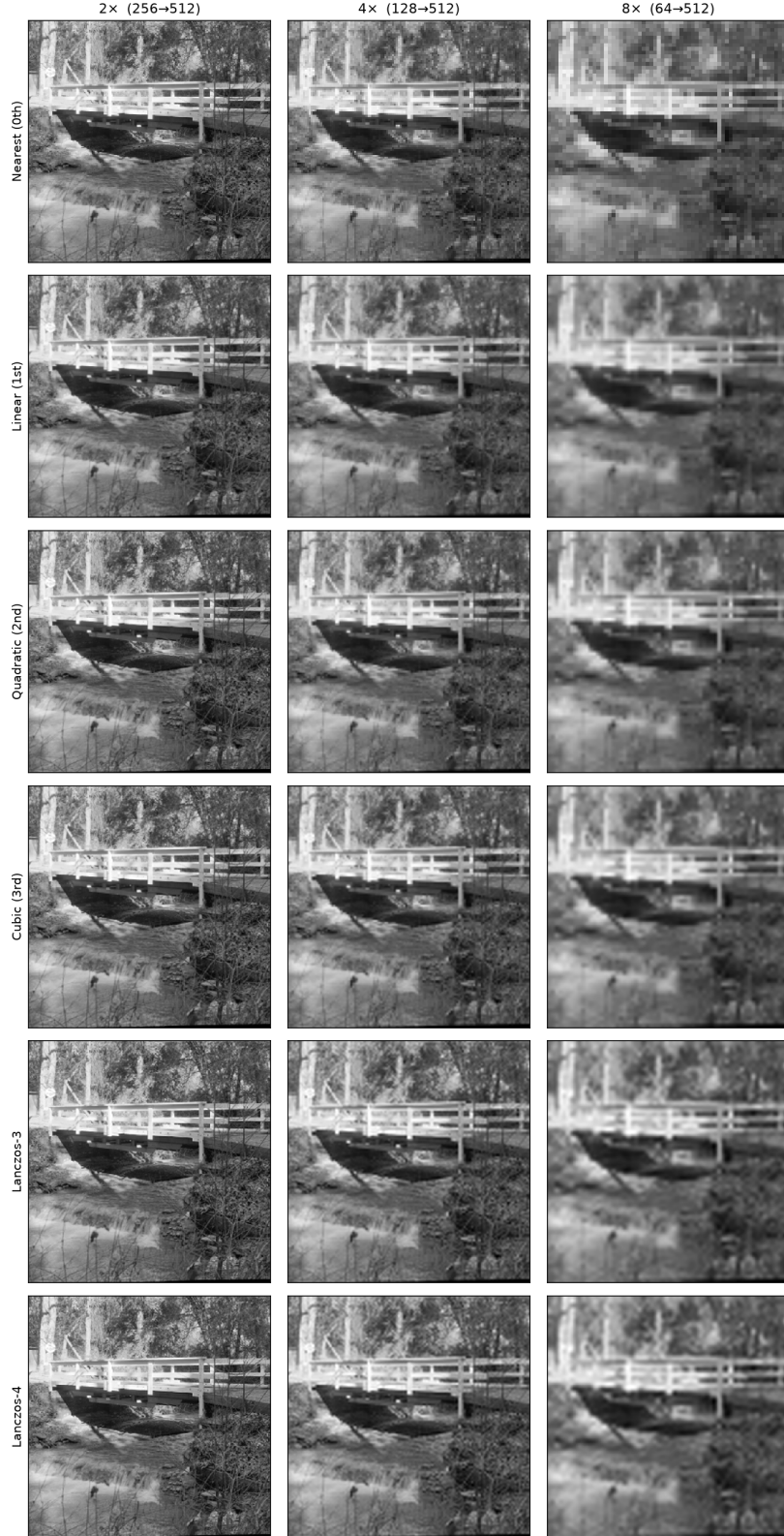


Figure 6: Super-resolved results for *stream and bridge*.

5.3 Magnified crops

Full-frame views cannot resolve two-pixel artefacts. The following are 64×64 regions magnified for display with nearest-neighbour screen rendering, so that what is shown is the output of the pipeline and not a re-interpolation introduced by the plotting library.

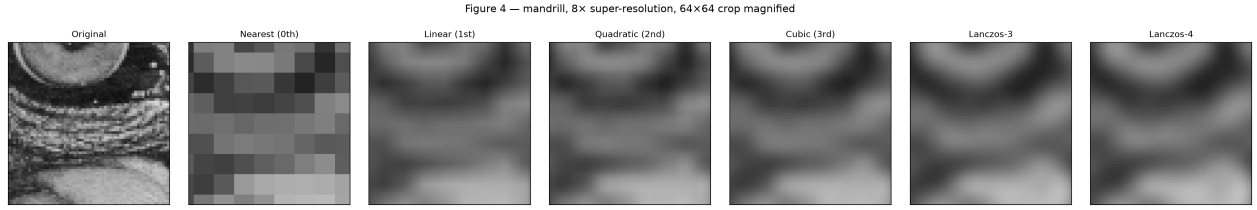


Figure 7: *Mandrill*, 8 $\times$, magnified crop. The original contains individual fur strands and a sharp eye boundary. Nearest neighbour reproduces the input as 8 $\times$ 8 constant tiles; every higher-order method produces a smooth blob. No method recovers a single strand.

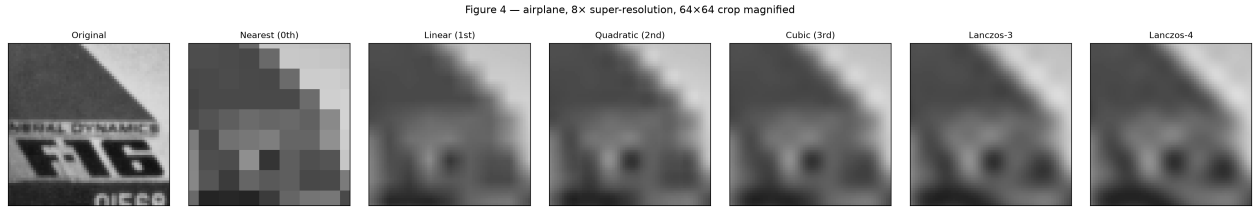


Figure 8: *Airplane*, 8 $\times$, magnified crop of the tail lettering against sky — the configuration in which ringing from the negative kernel lobes is visible.

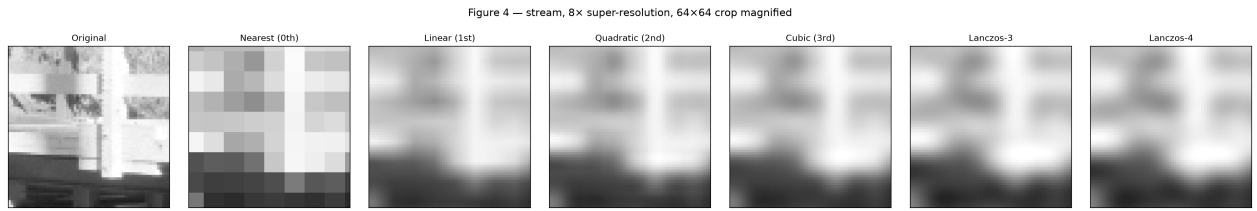


Figure 9: *Stream and bridge*, 8 $\times$, magnified crop of the railing against the dark background — staircase artefacts in nearest neighbour, halos in the higher orders.

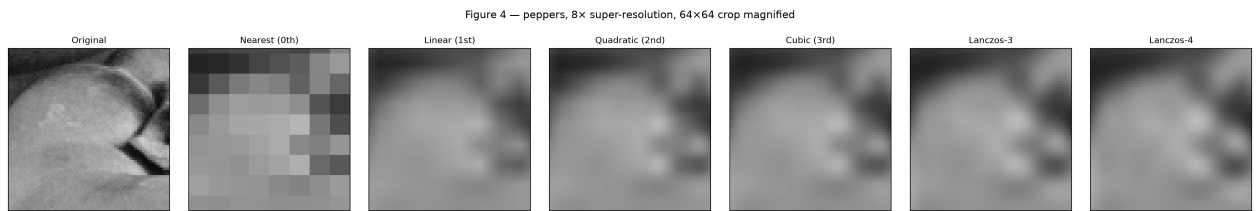


Figure 10: *Peppers*, 8 $\times$, magnified crop at a bright rim against a dark pepper body.

5.4 PSNR

Table 3: PSNR (dB) with respect to the original 512×512 grayscale image. Higher is better; best per column in bold.

Image	Method	$2\times$	$4\times$	$8\times$
Mandrill	Nearest (0th)	23.35	20.85	19.70
	Linear (1st)	23.17	20.89	19.80
	Quadratic (2nd)	23.74	21.06	19.89
	Cubic (3rd)	23.75	21.08	19.92
	Lanczos-3	23.83	21.09	19.94
	Lanczos-4	23.80	21.07	19.94
Airplane	Nearest (0th)	28.87	24.70	21.73
	Linear (1st)	30.11	25.49	22.22
	Quadratic (2nd)	31.09	25.99	22.52
	Cubic (3rd)	31.34	26.20	22.63
	Lanczos-3	31.85	26.46	22.75
	Lanczos-4	31.90	26.48	22.75
Peppers	Nearest (0th)	29.86	26.01	22.85
	Linear (1st)	30.98	27.21	23.88
	Quadratic (2nd)	31.64	27.66	24.32
	Cubic (3rd)	31.78	27.89	24.55
	Lanczos-3	32.03	28.12	24.79
	Lanczos-4	32.04	28.14	24.81
Stream	Nearest (0th)	25.68	22.27	20.28
	Linear (1st)	25.85	22.67	20.58
	Quadratic (2nd)	26.58	22.97	20.80
	Cubic (3rd)	26.65	23.06	20.86
	Lanczos-3	26.85	23.16	20.92
	Lanczos-4	26.83	23.16	20.91

5.5 SSIM

Table 4: SSIM with respect to the original 512×512 grayscale image. Higher is better; best per column in bold.

Image	Method	$2\times$	$4\times$	$8\times$
Mandrill	Nearest (0th)	0.7306	0.4510	0.2942
	Linear (1st)	0.6729	0.4254	0.2990
	Quadratic (2nd)	0.7296	0.4528	0.3086
	Cubic (3rd)	0.7307	0.4555	0.3101
	Lanczos-3	0.7428	0.4617	0.3125
	Lanczos-4	0.7419	0.4610	0.3120
Airplane	Nearest (0th)	0.9241	0.7935	0.6576
	Linear (1st)	0.9312	0.8175	0.6971
	Quadratic (2nd)	0.9463	0.8332	0.7055
	Cubic (3rd)	0.9484	0.8371	0.7078
	Lanczos-3	0.9526	0.8403	0.7072
	Lanczos-4	0.9524	0.8395	0.7061
Peppers	Nearest (0th)	0.8757	0.7551	0.6143
	Linear (1st)	0.8793	0.7967	0.6981
	Quadratic (2nd)	0.8919	0.8079	0.7067
	Cubic (3rd)	0.8926	0.8117	0.7121
	Lanczos-3	0.8939	0.8133	0.7136
	Lanczos-4	0.8928	0.8126	0.7126
Stream	Nearest (0th)	0.7838	0.5163	0.3281
	Linear (1st)	0.7556	0.5132	0.3462
	Quadratic (2nd)	0.8009	0.5425	0.3599
	Cubic (3rd)	0.8037	0.5474	0.3617
	Lanczos-3	0.8154	0.5555	0.3635
	Lanczos-4	0.8152	0.5554	0.3627

Table 5: PSNR and SSIM averaged over the four images. Best per column in bold.

Method	Mean PSNR (dB)			Mean SSIM		
	$2\times$	$4\times$	$8\times$	$2\times$	$4\times$	$8\times$
Nearest (0th)	26.94	23.46	21.14	0.8286	0.6290	0.4736
Linear (1st)	27.53	24.06	21.62	0.8098	0.6382	0.5101
Quadratic (2nd)	28.26	24.42	21.88	0.8422	0.6591	0.5202
Cubic (3rd)	28.38	24.56	21.99	0.8438	0.6629	0.5229
Lanczos-3	28.64	24.71	22.10	0.8512	0.6677	0.5242
Lanczos-4	28.64	24.71	22.10	0.8506	0.6671	0.5234

5.6 Runtime

Table 6: Runtime in milliseconds, averaged over the four images. Each per-image value is the median of 15 timed runs after three warm-up calls, and covers the interpolation call only. T is the number of taps per axis.

Method	T	$2\times$ (256 $\rightarrow$ 512)	$4\times$ (128 $\rightarrow$ 512)	$8\times$ (64 $\rightarrow$ 512)
Nearest (0th)	2	1.011 [†]	0.653	0.612
Linear (1st)	3	1.012	0.840	0.821
Quadratic (2nd)	4	1.309	1.100	1.054
Cubic (3rd)	5	1.552	1.361	1.289
Lanczos-3	7	2.057	1.758	1.666
Lanczos-4	9	2.786	2.269	2.093

[†] Inflated by one first-touch outlier (mandrill, 1.778 ms); the other three images give 0.74–0.77 ms.

Table 7: Quality versus cost, averaged over all images and all enlargement factors.

Method	Mean PSNR (dB)	Mean SSIM	Mean runtime (ms)	Runtime per tap (ms)
Nearest (0th)	23.85	0.6437	0.759	0.379
Linear (1st)	24.40	0.6527	0.891	0.297
Quadratic (2nd)	24.85	0.6738	1.154	0.289
Cubic (3rd)	24.98	0.6765	1.401	0.280
Lanczos-3	25.15	0.6810	1.827	0.261
Lanczos-4	25.15	0.6804	2.383	0.265

6 Discussion

6.1 Quality across enlargement factors

Quality falls monotonically from $2\times$ to $4\times$ to $8\times$ for every method on every image, with no exception across all 24 method–image combinations. Averaged over the four images, cubic drops from 28.38 dB at $2\times$ to 24.56 dB at $4\times$ and 21.99 dB at $8\times$ — 3.82 dB for the first step and a further 2.57 dB for the second. SSIM degrades more steeply in relative terms, from 0.844 to 0.663 to 0.523.

At $2\times$, one output sample in four coincides with a retained sample; at $8\times$ only one in sixty-four does, so 63/64 of the output is estimated. More fundamentally, the block average (3) is a low-pass filter whose cutoff scales as $1/f$, and the content it removes is not present in the low-resolution image in any form. Equation (5) reconstructs intensities *between* known samples; it does not invert a low-pass filter. Figure 7 makes this concrete.

6.2 Visual differences among the methods

Nearest neighbour is visibly distinct from everything else, replicating each input sample over an $s \times s$ block and producing hard tile boundaries that are unmistakable at $8\times$. Linear interpolation removes the blocking at the cost of blur, since each output is a convex combination of neighbours; it gains 0.59 dB over nearest at $2\times$ and 0.48 dB at $8\times$.

Beyond first order the methods converge. The step from linear to quadratic is worth 0.73 dB at $2\times$; quadratic to cubic 0.12 dB; cubic to Lanczos-3 0.26 dB. The total spread from quadratic to Lanczos-4 is 0.38 dB, against 1.70 dB from nearest to quadratic. Kernel order past the second therefore buys progressively less.

Notably, **Lanczos-3 equals or beats Lanczos-4 on SSIM in all twelve cells** and matches it on mean PSNR, at 23% lower cost. Widening the sinc window from $A = 3$ to $A = 4$ adds two

further taps per axis whose contribution is dominated by additional ringing rather than additional recovered detail. Greater support is not automatically better.

6.3 Artefacts

Blocking and staircase are confined to nearest neighbour and worsen with the enlargement factor; Figure 7 shows the 8×8 constant tiles directly. This follows from (10), which models intensity as constant over the region each sample represents.

Blur is present in all interpolated results and dominant at $8\times$. It is most damaging on textured content, and is the principal reason the mandrill scores poorly throughout.

Ringing and overshoot are restricted to kernels (12), (13) and (14), whose negative lobes drive the reconstruction beyond the range of the surrounding samples near a high-contrast edge, producing a bright band on the light side and a dark band on the dark side. These are visible in Figures 8 and 9. Kernels (10) and (11) are non-negative and, with the normalisation (9), cannot overshoot: their outputs are bounded by the convex hull of the contributing samples. All results were clipped by (2) before evaluation, which truncates the most extreme excursions but does not remove the halo.

6.4 Measured overshoot

Because the saved outputs are clipped by (2), the excursions are not directly observable in any of the figures above. They were therefore measured on the raw, unclipped output of (6), before rounding and clipping. Table 8 reports the fraction of output pixels falling outside $[0, 255]$ and the extreme values reached.

Table 8: Overshoot measured on the unclipped reconstruction. Percentages are the mean over the four images; the range columns give the extreme values reached anywhere in the study.

Method	Pixels outside $[0, 255]$ (%)			Extreme value reached	
	$2\times$	$4\times$	$8\times$	minimum	maximum
Nearest (0th)	0.0000	0.0000	0.0000	0.00	255.00
Linear (1st)	0.0000	0.0000	0.0000	0.00	254.94
Quadratic (2nd)	0.0396	0.0456	0.0067	-7.27	263.66
Cubic (3rd)	0.0672	0.0589	0.0155	-8.21	265.18
Lanczos-3	0.0690	0.0793	0.0535	-12.78	277.39
Lanczos-4	0.0859	0.0795	0.0584	-14.06	278.24

Three results follow.

The non-negativity argument is confirmed empirically. Nearest neighbour and linear interpolation produce *exactly* zero out-of-range pixels in all twelve image-factor cells, with observed minima never below 0 and maxima never above 255. This is the measured counterpart of the bound argued in Section 6.3: with non-negative kernels and the normalisation (9), the output is a convex combination of its input taps and therefore cannot leave their range.

Overshoot grows with kernel support. The extreme excursion widens monotonically from quadratic to Lanczos-4, which reaches 278.24 — more than 23 intensity levels beyond white — and -14.06 below black. Each additional pair of taps contributes a further negative lobe. This is the quantitative explanation for the result noted earlier that Lanczos-3 equals or beats Lanczos-4 on SSIM in all twelve cells: widening the window from $A = 3$ to $A = 4$ increases overshoot without a corresponding gain in recovered detail.

Overshoot requires both negative lobes and content near the range boundary. The per-image figures reveal a condition not visible in the averages. Mandrill, despite being the most edge-dense image in the set, produces essentially no overshoot at all — a single cell at 0.0004% and zero everywhere else — because its intensity range is only 12 to 214, so the ringing oscillations have headroom and never leave $[0, 255]$. Stream produces the most (0.33% at $2\times$ for Lanczos-4) because its original saturates at both 0 and 255, so any oscillation at those edges immediately exceeds the representable range. Negative kernel lobes cause *ringing*; whether that ringing manifests as *overshoot* depends on whether the underlying content sits near the boundary of the intensity range. The two are distinct, and only the latter is affected by clipping.

The dependence on enlargement factor is not monotone and differs by image. On stream the overshoot falls with the factor ($0.330 \rightarrow 0.266 \rightarrow 0.098\%$ for Lanczos-4), because block averaging (3) pulls saturated pixels away from the boundary as f grows. On peppers it rises ($0.013 \rightarrow 0.052 \rightarrow 0.134\%$), because larger blocks make adjacent low-resolution samples differ more sharply across an edge, strengthening the ringing itself. Both mechanisms are present; which dominates is a property of the image.

6.5 Quality versus runtime

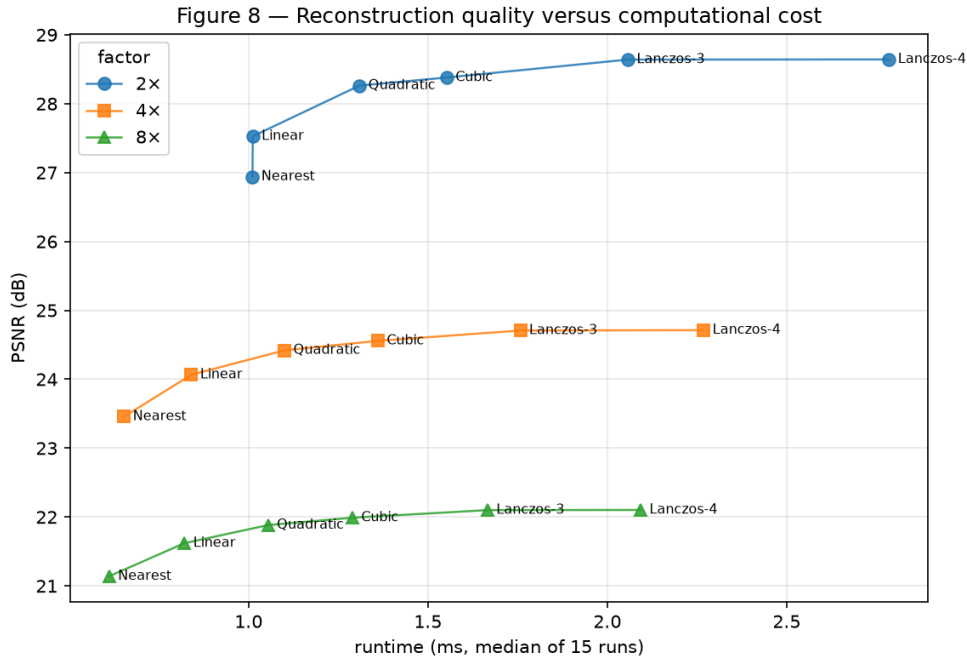


Figure 11: Reconstruction quality against computational cost, one curve per enlargement factor. The knee of every curve lies at quadratic interpolation.

All three curves in Figure 11 have the same shape: a steep initial rise followed by a long flat tail. At $2\times$, moving from nearest to quadratic buys 1.32 dB for 0.30 ms; moving from quadratic to Lanczos-4 buys a further 0.38 dB for 1.48 ms. The second purchase costs roughly five times as much for less than a third of the benefit. Quadratic or cubic interpolation sits at the knee, which is consistent with cubic convolution being the conventional default in practice.

Runtime is essentially independent of image content — the spread across the four images at fixed method and factor is within measurement noise — and varies only weakly with the enlargement factor, since the output size is fixed at 512×512 and the work is dominated by the number of output pixels rather than input samples.

6.6 Dependence on image content

Content affects the results in two distinct ways.

Absolute quality. At $8\times$, peppers reaches 24.81 dB while mandrill reaches only 19.94 dB — a gap of nearly 5 dB under identical processing. Smooth regions are well approximated by low-order polynomials, which is exactly what every kernel in Section 3.6 assumes. Dense texture violates that assumption.

Spread between methods. The gap between best and worst method is widest on edge-dominated content and narrowest on texture: 3.03 dB on airplane at $2\times$ (28.87 to 31.90) versus 0.24 dB on mandrill at $8\times$ (19.70 to 19.94). Where the signal has been destroyed, all six methods reconstruct from the same impoverished data and converge to nearly the same answer.

6.7 A case where nearest neighbour wins

On mandrill at $2\times$, nearest neighbour outperforms linear interpolation on both metrics: 23.35 versus 23.17 dB, and 0.7306 versus 0.6729 on SSIM. The same inversion appears on stream at $2\times$ (0.7838 versus 0.7556) and in the SSIM averages, where nearest neighbour beats linear at $2\times$ (0.8286 versus 0.8098).

This is not an error. At $2\times$, nearest neighbour reproduces every retained sample exactly and preserves local contrast, adding only blocking. Linear interpolation replaces interpolated pixels with convex combinations, reducing local variance everywhere. On an image whose content is almost entirely high-frequency texture, the variance loss costs more than the blocking does. SSIM penalises this more sharply than PSNR because the contrast and structure terms in (18) respond directly to σ_x^2 and σ_y^2 from (17), whereas PSNR sees only squared error. Section 7 identifies the mechanism precisely.

7 Additional investigation: region-wise error decomposition

The discussion above asserts that the advantage of higher-order interpolation is earned at edges. That claim was tested directly by splitting the error by local gradient magnitude.

7.1 Method

The gradient magnitude of the reference is computed with the Sobel operator, implemented from scratch as a pair of 3×3 correlations with

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = S_x^\top, \quad G = \sqrt{(S_x * I)^2 + (S_y * I)^2}. \quad (19)$$

Two disjoint masks are then defined by percentile thresholds on G : the *edge* mask is the top 10% of pixels by gradient magnitude, the *smooth* mask the bottom 50%. PSNR is recomputed over each mask alone,

$$\text{PSNR}_{\mathcal{M}} = 10 \log_{10} \left(\frac{L^2 |\mathcal{M}|}{\sum_{(m,n) \in \mathcal{M}} (I(m,n) - \hat{I}(m,n))^2} \right). \quad (20)$$

Figure 5 — Gradient-based region masks (white = selected)



Figure 12: Gradient-based region masks. The edge masks trace genuine structure on airplane, peppers and stream — the fuselage outline, the pepper boundaries, the bridge railing. On mandrill the mask is scattered across the whole frame, because the image has no edge/non-edge distinction to make.

7.2 Results

Table 9: PSNR (dB) computed separately over edge and smooth regions, averaged over the four images. Best per column in bold.

Method	Edge regions (top 10% gradient)			Smooth regions (bottom 50%)		
	2×	4×	8×	2×	4×	8×
Nearest (0th)	19.62	17.28	16.06	35.43	29.42	25.30
Linear (1st)	21.22	18.32	16.75	33.74	29.44	25.50
Quadratic (2nd)	21.72	18.53	16.85	34.62	29.94	26.01
Cubic (3rd)	21.92	18.64	16.89	34.55	30.13	26.24
Lanczos-3	22.30	18.79	16.95	34.41	30.18	26.41
Lanczos-4	22.36	18.80	16.96	34.29	30.15	26.39
Spread (best – worst)	2.74	1.52	0.90	1.69	0.77	1.11

Figure 6 — Gain over nearest neighbour, split by region type (mean over four images)

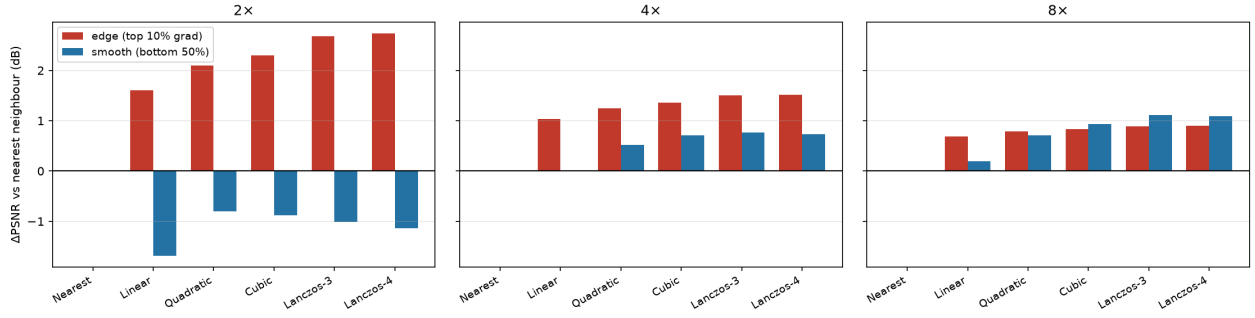


Figure 13: Gain over nearest neighbour, split by region type. At $2\times$ every method beats nearest at edges and *loses* to it in smooth regions — a complete crossover with no exceptions. By $8\times$ both bars are positive, and for cubic and both Lanczos kernels the smooth-region gain exceeds the edge gain.

7.3 Findings

A complete crossover at $2\times$. Nearest neighbour attains the *highest* smooth-region PSNR of all six methods at $2\times$ (35.43 dB) and the *lowest* edge-region PSNR (19.62 dB). The pattern holds on all four images individually — on airplane, 44.22 dB smooth against 19.77 dB edge. This resolves the anomaly of Section 6.7: nearest neighbour is not uniformly worse at $2\times$, it is the *best* method on flat regions and the *worst* on edges. Which effect dominates the global metric depends on how much of the image is flat, which is why the inversion appears on mandrill and stream but not on airplane or peppers.

The mechanism is visible in (10) versus (11). At $2\times$ nearest neighbour reproduces retained samples exactly and, within a flat region, the replicated block is a genuinely better estimate than a smoothed average of neighbours that all differ slightly by noise. At an edge, block replication is exactly the wrong model.

The crossover disappears with enlargement. By $4\times$ the smooth-region bars in Figure 13 have turned positive, and by $8\times$ the smooth-region gain (+1.11 dB for Lanczos-3) exceeds the edge gain (+0.90 dB). At high enlargement the 8×8 tiles produced by (10) are so coarse that they damage even flat regions, and the advantage of higher-order interpolation becomes universal rather than edge-specific.

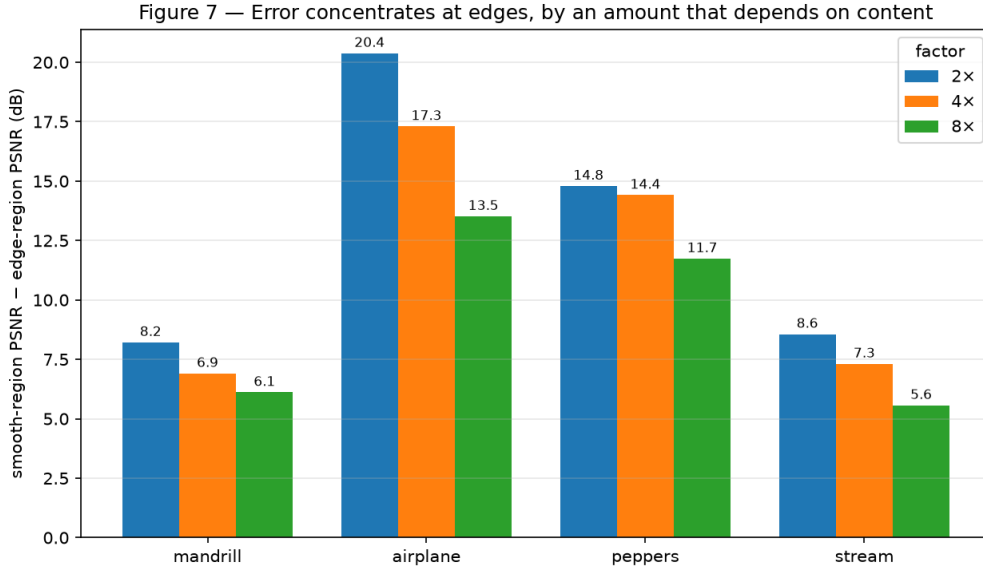


Figure 14: Difference between smooth-region and edge-region PSNR. The ordering tracks how much genuinely flat area each image contains.

Error concentrates at edges, by a content-dependent amount. The smooth-to-edge gap ranges from 20.4 dB on airplane at 2× down to 8.2 dB on mandrill, with peppers (14.8) and stream (8.6) in between — the same ordering as the amount of flat area in each image. Mandrill’s small gap is not evidence that it reconstructs edges well; it reflects that its “smooth” half is merely the less-textured half of a fully textured image, so neither region is reconstructed well and the two scores converge. The gap narrows with enlargement for every image, for the same reason the crossover disappears.

The method spread is an edge phenomenon. At 2× the spread between best and worst method is 2.74 dB in edge regions but only 1.69 dB in smooth regions — and the smooth-region spread is itself mostly the *penalty* incurred by the smoothing methods rather than a gain. Kernel choice matters where a genuine decision about the intensity profile must be made; in flat regions every kernel returns approximately the local mean regardless of its order.

8 Additional investigation: cost scaling of the separable engine

Figure 9 — Computational cost scales with kernel support

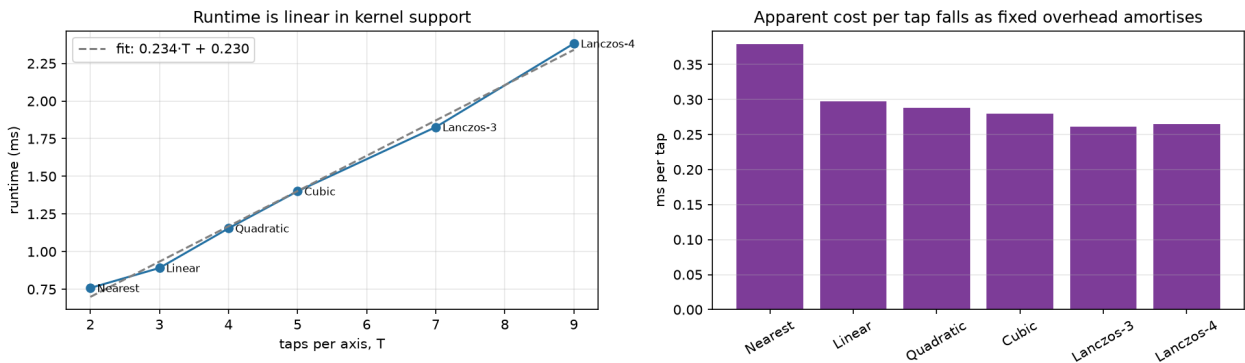


Figure 15: Measured runtime against taps per axis. Left: runtime is linear in T , as (6) predicts. Right: the apparent cost per tap falls with T because the fixed per-call overhead amortises.

Section 3.4 predicts that the separable formulation (6) costs $\mathcal{O}(2T)$ per output pixel rather than the $\mathcal{O}(T^2)$ of the direct form. Fitting the measured mean runtime against T gives

$$t(T) \approx 0.234T + 0.230 \quad \text{ms}, \quad (21)$$

which is linear in T across the full range $T = 2$ to $T = 9$, confirming the prediction. A direct $\mathcal{O}(T^2)$ implementation would have curved upward, costing roughly $4.5\times$ more than measured at $T = 9$.

The right panel of Figure 15 shows the apparent cost per tap falling from 0.379 ms at $T = 2$ to 0.261 ms at $T = 7$. This is not a violation of linearity: it is the constant term in (21). The 0.230 ms intercept is per-call overhead — array allocation, construction of the weight matrices, and tensor-contraction setup — which is independent of T . It accounts for 30% of the runtime of the nearest-neighbour kernel but only 10% of Lanczos-4’s. The *marginal* cost of an additional tap is constant at 0.234 ms, which is the quantity (6) actually predicts.

9 Conclusion

Across four images, three enlargement factors and six kernels implemented from first principles, five conclusions are supported.

Reconstruction quality degrades monotonically with the enlargement factor and cannot recover frequencies removed by the downsampling filter (3). The largest quality gain comes from moving off zeroth order; gains beyond second or third order are small while cost continues to grow linearly with kernel support. Each kernel family carries a characteristic artefact — blocking for the zeroth order, blur for the first, ringing for the negative-lobe kernels — so the choice among them is a trade rather than a strict ordering, as Lanczos-3 outscoring Lanczos-4 demonstrates. Direct measurement of the unclipped output confirms that the non-negative kernels never leave $[0, 255]$ while Lanczos-4 exceeds it by up to 23 intensity levels, and shows that overshoot requires content near the boundary of the intensity range in addition to negative kernel lobes.

The region-wise decomposition shows that these effects are not uniform over the image. At $2\times$, higher-order interpolation improves edges and actively *degrades* smooth regions relative to nearest neighbour, a complete crossover across all four images that explains why nearest neighbour wins outright on texture-dominated content. Only at high enlargement does higher-order interpolation become universally beneficial.

Finally, the measured runtime confirms the $\mathcal{O}(2T)$ cost of the separable formulation, with a constant marginal cost of 0.234 ms per tap and a fixed 0.230 ms per-call overhead.

Reproducibility

All results were generated by the accompanying Python notebook. Per-configuration values are written to `results/metrics.csv`, `results/metrics_regions.csv` and `results/metrics_overshoot.csv` (72 rows each); every table in this report is derived from those files. The pipeline contains no stochastic component and reproduces bit-identically on re-run. NumPy is used for array arithmetic, Pillow for TIFF reading, pandas for tabulation and Matplotlib for figures; no library implementation of any algorithm described in Section 3 is used.